

飞虹 Code 技术说明书

文档版本：V8.0.0

发布日期：2026-09-03

适用版本：v8.0.0

技术栈：TypeScript + Node.js + Express + Electron + Capacitor + SQLite

一、项目概述

1.1 产品定位

飞虹 Code (feihong-code) 是一款终端 AI 编程智能体，对标 Muse Code / Cursor CLI / Claude Code，具备多模型路由、企业级 RBAC 审计、全自动 SWE Agent、离线私有化部署等能力。v7.9.0 引入 Hermes Agent 自我进化框架，实现持久记忆、自演化技能、自动化调度三大核心能力。

1.2 核心特性

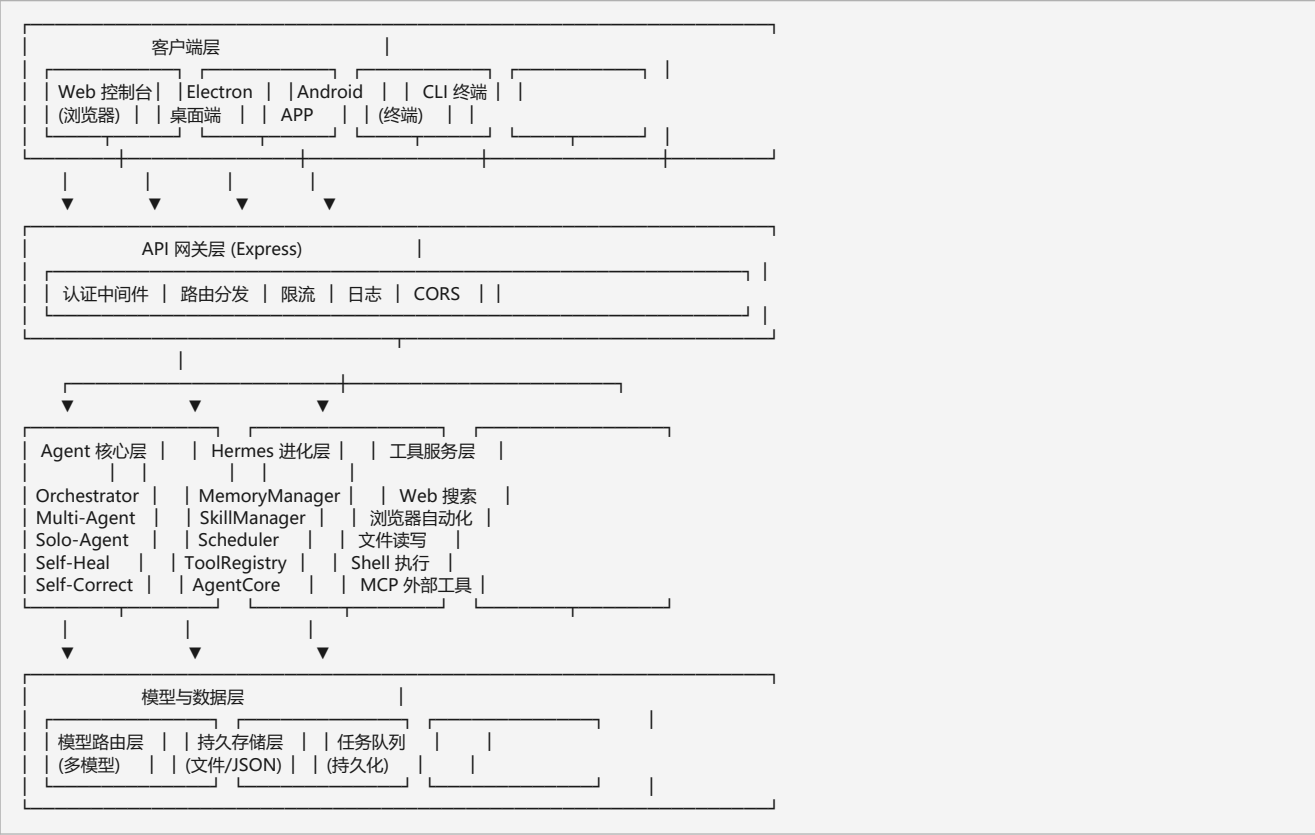
特性	说明
多模型路由	DeepSeek / 通义 / Ollama / OpenAI / Agnes / 自定义 OpenAI 兼容
企业级审计	RBAC 权限控制、操作审计日志
全自动 SWE Agent	自动创建技能、改进技能、持久记忆、FTS5 跨会话搜索
离线私有化	支持本地 Ollama、内网部署、无云依赖
Hermes 自我进化	持久记忆 + 自演化技能 + 自动化调度 + 工具集
免密网络层	5节点环形轮转 + Ring Failover，开箱即用
SQLite 统一存储 (v8.0)	10 张表，模型 Key 加密落盘，零外部依赖
Docker 沙盒 (v8.0)	容器隔离执行，网络/资源/命令三重限制
Honcho 语义记忆 (v8.0)	用户建模 + 事实记忆 + 跨会话检索 + 记忆注入
多端覆盖	Web 控制台 + Electron 桌面端 + Android APP
多平台桥接	微信 / 飞书 / 元宝（配置后启用）

1.3 技术指标

指标	值
源代码文件数	160+
子系统数量	21 个
核心依赖	5 个 (express / monaco-editor / playwright-core / zod / @capacitor/core)
后端编译产物	dist/ 目录, server.js 130KB
移动端 APK	3.2MB (v7.9.0)
健康检查响应	平均 14.6ms
首页加载	59ms (74KB HTML)
SQLite 存储	feihong.db, SCHEMA_VERSION=1, 10 张表
集成测试	34 用例 100%

二、系统架构

2.1 整体架构图



2.2 分层架构说明

层级	职责	核心模块
客户端层	用户交互界面	Web 控制台 / Electron / Android APP / CLI
API 网关层	请求路由、认证、限流、日志	Express server.ts (122KB) 、auth.ts
Agent 核心层	任务编排、多 Agent 协作、自我修复	orchestrator、multi-agent、self-heal
Hermes 进化层	持久记忆、自演化技能、自动化调度	hermes-agent.js (移动端)、self-evolve (后端)
工具服务层	外部工具调用、沙盒执行	tools/ (22个文件) 、sandbox、mcp
模型与数据层	模型路由、持久化存储、任务队列	models/、memory/、task-queue

三、目录结构

3.1 后端源码结构 (src/)

```

src/
├── agent/          # Agent 核心 (38个文件)
│   ├── orchestrator.ts    # 统一编排器 (36KB)
│   ├── multi-agent.ts     # 多 Agent 协作
│   ├── solo-agent.ts      # 单 Agent
│   ├── custom-agent.ts    # 自定义 Agent
│   ├── team.ts            # 团队协作
│   ├── team-collaboration.ts # 团队协作扩展
│   ├── self-heal.ts       # 自我修复
│   ├── self-correction.ts  # 自我纠错
│   ├── self-improver.ts   # 自我改进
│   ├── event-driven-agent.ts # 事件驱动 Agent
│   ├── layered-memory.ts  # 分层记忆
│   ├── context-compactor.ts # 上下文压缩
│   ├── experience.ts       # 经验沉淀
│   ├── completion-engine.ts # 补全引擎 (27KB)
│   ├── code-writer.ts     # 代码生成
│   ├── code-review.ts     # 代码审查
│   ├── code-rag.ts        # 代码 RAG
│   ├── git-integration.ts  # Git 集成
│   ├── repo-reader.ts     # 仓库读取
│   ├── repo-context.ts    # 仓库上下文
│   ├── symbol-index.ts    # 符号索引
│   ├── quality-gate.ts    # 质量门控
│   ├── planner.ts         # 规划器
│   ├── prompts.ts         # 提示词管理
│   ├── swe-agent.ts       # SWE Agent
│   ├── swe-planner.ts     # SWE 规划
│   ├── swe-verifier.ts    # SWE 验证
│   ├── subagent.ts        # 子 Agent
│   ├── subagent-summary.ts # 子 Agent 摘要
│   ├── parallel-orchestrator.ts # 并行编排
│   ├── change-manager.ts  # 变更管理
│   ├── browser-agent.ts   # 浏览器 Agent
│   ├── design-to-code.ts  # 设计转代码
│   ├── lint.ts            # Lint 检查
│   ├── type-checker.ts    # 类型检查
│   └── context-budget.ts  # 上下文预算
├── cli/            # CLI 入口 (9个文件)
│   ├── index.ts        # 主入口
│   ├── run.ts          # 运行逻辑 (63KB)
│   ├── commands.ts     # 命令定义
│   ├── version.ts      # 版本号集中定义
│   ├── tui.ts          # 终端 UI
│   ├── tui-run.ts      # TUI 运行
│   ├── repl.ts         # REPL 交互
│   └── multi-pane-tui.ts # 多面板 TUI
├── web/            # Web 服务器 (16个文件)
│   ├── server.ts       # Express 服务器 (122KB)
│   ├── auth.ts         # 认证
│   ├── channels.ts     # 多通道
│   ├── task-queue.ts   # 任务队列 (23KB)
│   ├── extra-apis.ts   # 扩展 API
│   ├── web-config.ts   # Web 配置
│   ├── express.d.ts    # Express 类型扩展
│   └── public/         # Web 前端静态资源
├── tools/          # 工具集 (22个文件)
│   ├── index.ts        # 工具注册中心
│   ├── tool.interface.ts # 工具接口
│   ├── tool.registry.ts # 工具注册表
│   ├── sandbox.ts      # 沙盒执行
│   ├── safe-path.ts    # 安全路径
│   ├── search/         # 网页搜索
│   ├── shell/          # Shell 执行
│   ├── file/           # 文件读写
│   ├── browser/        # 浏览器自动化 (8个工具)
│   ├── mcp/            # MCP 外部工具
│   ├── analysis/       # 分析工具
│   ├── generator/      # 生成工具
│   ├── skills/         # 技能工具
│   ├── verify/         # 验证工具
│   └── web/            # Web 工具
├── memory/         # 记忆系统 (2个文件)
│   ├── index.ts        # 记忆管理
│   └── auto-summarize.ts # 自动摘要
├── skills/         # 技能系统 (7个文件)
│   ├── skill-loader.ts  # 技能加载器
│   ├── skill-market.ts  # 技能市场 (10KB)
│   ├── pua-hooks.ts    # PUA 扩展 hooks
│   ├── self-heal.ts     # 技能自愈
│   ├── goal.ts         # 目标技能
│   ├── grill.ts        # 追问技能
│   └── plan.ts         # 规划技能
├── self-evolve/    # 自我进化 (4个文件)
│   ├── manager.js       # 进化管理器 (17KB)
│   ├── manager.d.ts     # 类型定义
│   ├── hook.ts          # 进化钩子
│   └── self-heal-scheduler.ts # 自愈调度器
├── models/         # 模型管理 (7个文件)
├── enterprise/     # 企业版 (6个文件)
├── harness/        # 测试框架 (7个文件)
├── integrations/   # 第三方集成 (7个文件)
├── knowledge/      # 知识库 (1个文件)
├── lsp/            # 语言服务器 (2个文件)
├── plugins/        # 插件系统 (2个文件)
├── runtime/        # 运行时 (6个文件)
├── shared/         # 共享模块 (7个文件)
├── training/       # 训练 (2个文件)
├── voice/          # 语音 (1个文件)
└── workers/       # Worker (1个文件)

```

3.2 移动端源码结构 (app-mobile/)

```
app-mobile/
├── index.html      # 主页面 (23KB)
├── css/
│   └── style.css   # 样式 (21KB)
├── js/
│   ├── app.js      # 主应用逻辑 (83KB)
│   ├── game-templates.js # 12款游戏模板 (68KB)
│   ├── keyless.js  # 免密网络层 (16KB)
│   └── hermes-agent.js # Hermes Agent 核心 (24KB)
```

3.3 安卓工程结构 (android/)

```
android/
├── app/
│   ├── build.gradle # 构建配置 (versionCode=26, versionName=7.9.0)
│   ├── fhcode-release.jks # 签名密钥
│   └── src/main/
│       ├── AndroidManifest.xml
│       ├── java/ # Java 源码
│       └── assets/public/ # Web 资源 (与 app-mobile 同步)
├── build.gradle
├── gradle.properties
├── settings.gradle
└── gradlew.bat # Gradle Wrapper
```

四、核心模块技术设计

4.1 Hermes Agent 核心 (移动端 hermes-agent.js)

4.1.1 MemoryManager — 持久记忆系统

设计目标：实现跨会话长期记忆，维护 MEMORY.md 和 USER.md，自动摘要历史任务。

数据结构：

```
memory = {
  content: string, // MEMORY.md 完整内容
  updatedAt: number // 最后更新时间戳
}
user = {
  content: string, // USER.md 完整内容
  updatedAt: number
}
history = [ // 历史任务摘要 (最多100条)
  { time, type, summary, msgCount }
]
contexts = { // 项目上下文键值存储
  [key]: { value, time }
}
```

核心方法：

方法	功能
load()	从 localStorage 加载所有记忆数据
save()	持久化到 localStorage
getContextPrompt()	生成记忆注入提示词（拼接到系统提示）
digestConversation(messages, taskType)	对话结束后自动沉淀记忆
recall(query, limit)	关键词检索历史记忆
extractKeyFacts(messages)	从对话中提取关键事实（项目名、版本、技术栈）
extractUserPreferences(messages)	从对话中提取用户偏好
appendToSection(section, items, isUser)	向 MEMORY.md/USER.md 指定章节追加内容

记忆注入流程：

```
用户输入 → AgentCore.processInput()
→ MemoryManager.getContextPrompt()
→ 提取 MEMORY.md 项目上下文
→ 提取 USER.md 用户偏好
→ 列出最近5条历史任务
→ 拼接到系统提示词
→ 发送给大模型
```

自动沉淀触发点：每轮对话结束（callModelStream 的 onDone 回调），调用 digestConversation()。

4.1.2 SkillManager — 自演化技能系统

设计目标：从执行过的任务自动提炼可复用技能，兼容 agentskills.io 开放标准。

技能数据结构（兼容 agentskills.io）：

```
skill = {
  id: string,      // 唯一标识
  name: string,    // 技能名称
  description: string, // 技能描述
  trigger: string, // 触发词 (| 分隔多个)
  prompt: string,  // 提示词模板 (支持 {{content}} {{变量}})
  tools: string[], // 依赖工具列表
  tags: string[],  // 标签
  version: string, // 版本号
  createdAt: number, // 创建时间
  useCount: number, // 使用次数
  builtin?: boolean, // 是否内置
  autoExtracted?: boolean // 是否自动提炼
}
```

内置技能（4个）：

技能	触发词	用途			
内容摘要	摘要\	总结\	概括\	提炼	结构化摘要长文本
中英互译	翻译\	translate\	英文\	中文	中英文双向翻译
代码审查	代码审查\	code review	安全性/性能/规范 审查		
小红书文案	小红书\	种草文案	小红书风格种草文案		

技能匹配算法：

```
输入文本 → 小写化
→ 遍历所有技能
→ 触发词匹配 (+20分/词)
→ 技能名匹配 (+15分)
→ 描述匹配 (+5分)
→ 标签匹配 (+8分/标签)
→ 取最高分技能
→ 分数 >= 15 则匹配成功
```

技能执行：execute(skill, content, vars) 将 {{content}} 替换为用户输入，{{变量}} 替换为传入变量。

自动提炼：extractFromTask(messages, taskName) 从对话中提取用户需求和助手回复，生成新技能。

4.1.3 Scheduler — 自动化调度

设计目标：自然语言设定定时任务，替代手写 cron。

时间解析支持：

表达	解析结果	重复模式
每天 9:00	明天9:00	daily
每周一 10:30	下周一10:30	weekly
每隔 30 分钟	30分钟后	interval
14:00	今天/明天14:00	once

任务数据结构：

```
task = {
  id: string,
  name: string,
  expression: string,    // 原始自然语言表达
  prompt: string,       // 任务内容
  schedule: {
    timestamp: number,
    repeat: 'once'|'daily'|'weekly'|'interval',
    nextRun: number,    // 下次运行时间戳
    intervalMs?: number, // 间隔模式的毫秒数
    weekday?: number    // 周模式的星期几
  },
  enabled: boolean,
  createdAt: number,
  lastRun: number|null,
  runCount: number
}
```

调度机制：

- 每 30 秒检查一次到期任务 (setInterval)
- 到期任务触发 hermes-scheduled-task 自定义事件
- 重复任务自动计算下次运行时间
- 一次性任务执行后自动禁用

4.1.4 ToolRegistry — 工具集

设计目标：统一工具注册和调用机制。

已注册工具：

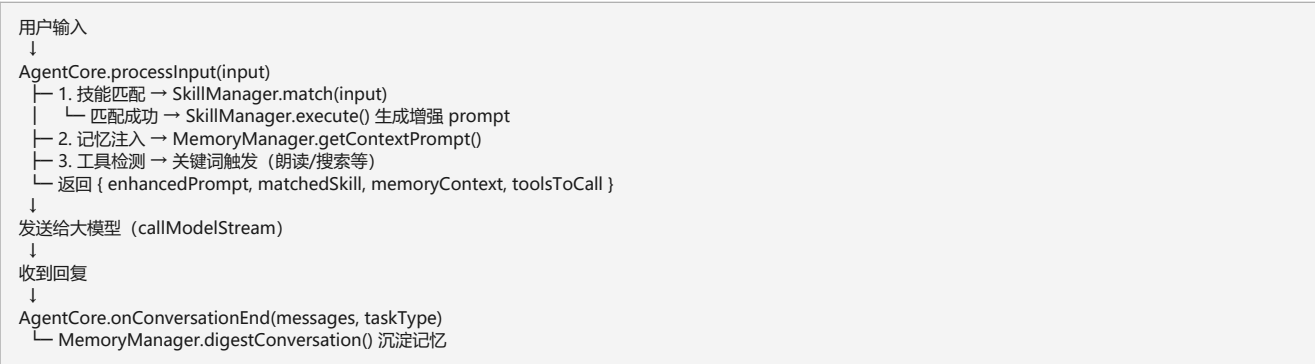
工具	功能	实现
tts	文字转语音	浏览器 SpeechSynthesis API
tts_stop	停止语音	speechSynthesis.cancel()
web_search	免密网页搜索	复用 keyless.js 双源搜索
read_file	文件读取	占位实现
recall_memory	记忆查询	MemoryManager.recall()

工具调用接口：

```
ToolRegistry.register(name, handler, description) // 注册工具
ToolRegistry.execute(name, args)                // 调用工具
ToolRegistry.list()                             // 列出所有工具
```

4.1.5 AgentCore — 统一执行循环

处理流程：



4.2 免密网络层 (keyless.js)

设计目标：内置 5 家厂商免费模型接口，环形轮转池，开箱即用。

5节点环形池：

节点	模型	权重	来源
ddg-gpt4o	gpt-4o-mini	30	DuckDuckGo AI
ddg-claude	claude-3-haiku	25	DuckDuckGo AI
ddg-llama	Llama 3.1 70B	25	DuckDuckGo AI
ddg-mixtral	Mixtral 8x7B	15	DuckDuckGo AI
wiki-knowledge	Wikipedia 知识引擎	5	Wikipedia API

Ring Failover 机制：

- 失败自动轮转下一家
- 连续失败 3 次冷却 30 秒
- 成功后固定在该节点
- 全部失败则友好提示配置自定义模型

DuckDuckGo AI 调用流程：

1. GET https://duckduckgo.com/duckchat/v1/status → 获取 x-vqd-4 token
2. POST https://duckduckgo.com/duckchat/v1/chat Header: x-vqd-4: <token> Body: { model, messages }
3. 流式解析响应

免密搜索：DuckDuckGo HTML 搜索 + Wikipedia API 双源并行。

4.3 后端 Web 服务器 (server.ts)

设计目标：Express 全功能 Web 服务器，支持 API、静态资源、WebSocket、认证。

核心中间件链：

请求 → CORS → 日志 → 认证(可选) → 限流 → 路由分发 → 响应
--

主要路由：

路由	方法	认证	功能
/api/health	GET	否	健康检查
/api/version	GET	是	版本信息
/api/status	GET	是	系统状态
/api/config	GET/POST	是	配置管理
/api/models	GET	是	模型列表
/api/agents	GET/POST	是	Agent 管理
/api/skills	GET/POST	是	技能管理
/api/tools	GET	是	工具列表
/api/tasks	GET/POST	是	任务队列
/api/knowledge	GET/POST	是	知识库
/api/plugins	GET/POST	是	插件管理
/	GET	否	Web 控制台首页

路由	方法	认证	功能
/*	GET	否	静态资源

认证机制：

- FH_WEB_TOKEN 环境变量生成访问令牌
- 未认证请求返回 401
- 令牌在启动日志中输出

任务队列：

- 持久化到 ~/.feihong-code/tasks/
- 启动时自动恢复（98个任务）
- 支持异步任务执行

五、数据存储设计

5.1 后端存储

数据	存储方式	路径
任务队列	JSON 文件	~/.feihong-code/tasks/
知识库	Markdown 文件	~/.feihong-code/knowledge/
插件	目录	~/.feihong-code/plugins/
自我进化	JSON 文件	~/.feihong-code/self-evolve/
日志	文本文件	FH_LOG_DIR 环境变量
配置	.env / JSON	项目根目录
SQLite 统一存储 (v8.0)	SQLite 数据库	\$FH_HOME/feihong.db (可 FH_DB_PATH 覆盖)

5.2 SQLite 数据存储 (v8.0 新增)

基于 Node.js 内置 node:sqlite (DatabaseSync) ， 零外部依赖， 10 张表：

表名	用途
meta	Schema 版本管理 (SCHEMA_VERSION=1, 自动迁移)
kv	通用键值 (models/config/settings)
tasks	任务持久化 (type/status/progress/input/result/error/model)
models	模型配置 (api_key AES-256-GCM 加密落盘)
agents	Agent 注册表
skills	技能库 (名称/触发词/提示词/使用计数)
memory	记忆键值 (MEMORY.md 等, kind 区分)
memory_history	会话摘要历史 (跨会话检索)
users + user_memory	Honcho 用户建模 (偏好/特质/事实记忆+重要度)
knowledge	知识库文档索引

安全特性：

- 敏感字段自动加密：models.api_key、config.value 使用主密钥（FH_SECRET 或 \$FH_HOME/.secret）AES-256-GCM 加密，格式 v1:iv:cipher:tag
- WAL 日志模式 + 外键约束
- 启动自动建表 + 版本化迁移，兼容已有库升级

5.3 移动端存储 (localStorage)

Key	用途
fh.app.models	模型列表
fh.app.defaultModelId	默认模型
fh.app.tasks	对话任务
fh.app.theme	主题
fh.app.games	游戏数据
fh.app.gamebest.<key>	游戏最高分
fh.app.gamerecent	最近玩过的游戏
fh.app.flashapps	闪应用
fh.app.vlconfig	视觉模型配置
fh.app.creative	创作配置
fh.hermes.memory.md	Hermes MEMORY.md
fh.hermes.user.md	Hermes USER.md
fh.hermes.history	Hermes 历史任务
fh.hermes.contexts	Hermes 项目上下文
fh.hermes.skills	Hermes 技能库
fh.hermes.scheduled	Hermes 定时任务

六、安全机制

6.1 认证与授权

- Web 控制台：FH_WEB_TOKEN 令牌认证
- API 接口：除 /api/health 外全部需要认证
- 企业版：RBAC 角色权限控制

6.2 沙盒执行

- Shell 命令在沙盒中执行 (sandbox.ts)
- 安全路径检查 (safe-path.ts)，防止路径遍历
- 可配置 FH_SHELL_ALLOW 白名单
- FH_REQUIRE_APPROVAL 审批机制
- **Docker 容器沙盒 (v8.0)**：src/tools/docker-sandbox.ts 实现 container 模式真实执行
 - docker run --rm 一次性容器，执行完自动销毁
 - 默认 --network none 禁用网络（可 FH_SANDBOX_NETWORK=true 开启）

- 工作区默认只读挂载（可 FH_SANDBOX_READONLY=false 关闭）
- 内存/CPU 资源限制（默认 512m / 1 CPU）
- 危险命令拦截（rm -rf /、mkfs、dd 写块设备、fork 炸弹）
- 超时强杀（默认 60s, docker kill）

6.3 数据安全

- .env 文件包含密钥，已加入 .gitignore
- API Key 在前端 localStorage 存储（移动端）
- 健康检查不返回敏感信息
- 不存在的 API 返回 401（不暴露存在性）

6.4 依赖安全

- 核心依赖仅 5 个，减少攻击面
- zod 用于输入验证
- playwright-core 用于浏览器自动化隔离

七、部署方式

7.1 后端部署

本地开发

```
npm install
npx tsc
$env:FH_WEB_PORT="8080"
node dist/cli/index.js serve
```

一键启动

```
node start-web.js
```

Docker 部署

```
docker build -t feihong-code .
docker run -p 8080:8080 feihong-code
```

Docker Compose

```
docker-compose up -d
```

7.2 移动端部署

APK 构建

```
cd android
$env:JAVA_HOME = "C:\Program Files\Microsoft\jdk-21.0.12-hotspot"
.\gradlew.bat assembleRelease --no-daemon
# 产物: android/app/build/outputs/apk/release/app-release.apk
```

签名配置

- Keystore: android/fhcode-release.jks
- storePassword: fhcode2026
- keyAlias: fhcode
- keyPassword: fhcode2026

7.3 Electron 桌面端

npm run electron:dev # 开发模式
npm run electron:build # 构建 Windows 安装包

八、API 接口规范

8.1 健康检查（公开）

请求: GET /api/health

响应:

```
{
  "ok": true,
  "product": "飞虹 Code",
  "version": "7.9.0",
  "signature": "晋江市飞虹智科技企业管理有限公司 · 飞扬企源研发中心 · 负责人: 吴赐虹",
  "enterprise": true,
  "lang": "zh",
  "wechat": "disabled",
  "feishu": "disabled",
  "yuanbao": "disabled",
  "time": "2026-09-03T03:22:41.905Z"
}
```

8.2 认证方式

所有受保护接口需要在请求头中携带令牌:

Authorization: Bearer <FH_WEB_TOKEN>

未认证响应:

```
{
  "error": "Unauthorized",
  "code": 401
}
```

8.3 错误响应格式

```
{
  "error": "错误描述",
  "code": 404,
  "details": {}
}
```

九、性能指标

9.1 后端性能（实测）

指标	值	评级
健康检查平均响应	14.6ms	优秀

指标	值	评级
健康检查最快响应	12ms	优秀
健康检查最慢响应	25ms	优秀
首页加载	59ms	优秀
首页大小	74KB	适中
并发稳定性	5/5 成功	优秀
内存占用	228MB	适中
启动时间	< 1秒	优秀

9.2 构建性能

指标	值
TypeScript 编译	< 5秒
APK 构建	17-23秒
APK 大小	3.2MB

十、扩展与集成

10.1 多平台桥接（需配置）

平台	环境变量	状态	
微信（企业微信/公众号）	`FH_WECHAT_MODE=wecom`	mp`	disabled
飞书	FH_FEISHU_ENABLED=true	disabled	
元宝	FH_YUANBAO_ENABLED=true	disabled	

10.2 MCP 外部工具

- 支持 Model Context Protocol 接入外部工具
- 工具目录：src/tools/mcp/
- 可扩展自定义 MCP 服务器

10.3 插件系统

- 动态插件加载
- 插件目录：~/.feihong-code/plugins/
- 支持自定义插件开发

十一、版本历史

版本	日期	主要变更
v7.6.0	2026-08-30	基础版本，20个子系统

版本	日期	主要变更
v7.7.0-v7.7.6	2026-08-31	移动端多轮对话修复、12款游戏、创作中心、布局修复
v7.8.0	2026-09-02	免密网络层（Keyless Web Tier） + Hermes 记忆基础
v7.9.0	2026-09-03	Hermes Agent 完整框架（持久记忆+自演化技能+自动化调度+工具集），后端版本对齐
V8.0.0	2026-09-03	文档版本升级至 V8.0.0，与 v8.0 产品能力（SQLite 统一存储 / Docker 沙盒 / Honcho 语义记忆）对齐

十二、联系方式

- **项目负责人：**吴赐虹
- **研发团队：**晋江市飞虹智科技企业管理有限公司 · 飞扬企源研发中心
- **技术支持：**通过飞虹 Code Web 控制台提交反馈

本文档随版本更新而维护，最新版本以项目仓库 docs/ 目录为准。